Q-learning 與 SARSA 演算法之比較研究 — Agent 對話紀錄

Q-learning vs SARSA 實驗開發全程對話
日期：2026-04-15

一、作業需求與環境建立

[1] USER：

一、作業目的
本作業旨在實作並比較兩種經典強化學習演算法——Q-learning 與 SARSA，透過相同環境與參數設定，分析其學習行為、收斂特性以及最終策略差異。

二、環境描述
本實驗採用經典的格子世界（Gridworld）問題，亦稱為「Cliff Walking」環境。環境設定如下：
- 使用一個矩形網格，可以讓使用者自行設定網格大小（例如 4 × 12）
- 起點（Start）位於左下角
- 終點（Goal）位於右下角
- 起點與終點之間的底部區域為「懸崖（Cliff）」
- 當代理（agent）進入懸崖時，會受到重大懲罰並回到起點。

三、問題設定
- 狀態空間（State Space）：所有網格位置
- 動作空間（Action Space）：上、下、左、右
- 獎勵機制（Reward）：每移動一步 −1，掉入懸崖 −100 並回到起點，到達終點回合結束
- 策略（Policy）：ε-greedy（例如 ε = 0.1）
- 學習率（α）：例如 0.1
- 折扣因子（γ）：例如 0.9
- 訓練回合數（Episodes）：至少 500 回合

四、作業內容
（一）演算法實作：Q-learning（離策略方法）、SARSA（同策略方法）
（二）訓練過程：在相同的環境與參數設定下，使用 ε-greedy 策略進行訓練
（三）結果分析：學習表現、策略行為、穩定性分析

[1] AGENT：

收到作業需求。我將使用純前端技術（HTML/CSS/JavaScript）開發一個 Web 應用程式來實作此作業。

【執行步驟】
1. 建立專案目錄 hw1-2/，建立 index.html、style.css、script.js 三個檔案
2. 在 script.js 中實作 CliffWalkingEnv 環境類別（configurable rows×cols grid）
3. 實作 Q-learning（off-policy）更新規則：Q(s,a) ← Q(s,a) + α[r + γ·max_a' Q(s',a') - Q(s,a)]
4. 實作 SARSA（on-policy）更新規則：Q(s,a) ← Q(s,a) + α[r + γ·Q(s',a') - Q(s,a)]
5. 使用 Canvas 繪製學習曲線圖，DOM 繪製策略方向格子
6. 加入深色玻璃態 (Glassmorphism) UI 主題

【結果】
成功建立了完整的 Cliff Walking 應用程式，包含：
- 可調整的參數面板（α, γ, ε, episodes, grid size）
- 雙演算法同時訓練與比較
- Canvas 繪製的平滑獎勵曲線圖
- 兩個併排的策略網格（箭頭方向 + 最佳路徑高亮）
- 自動生成的三段式分析報告（學習表現、策略行為、穩定性分析）

二、分析功能強化

💬 [2] USER：

（系統自動觸發）強化分析區段，加入收斂速度偵測、標準差、懸崖掉落次數等穩定性指標。

🤖 [2] AGENT：

已強化分析區段為三大面向：
📈 一、學習表現：收斂回合數（50-ep 移動平均 > -50）、最後 100 回合平均獎勵
⛰️ 二、策略行為：路徑長度比較、冒險 (Risky) / 保守 (Safe) 標籤、Off-policy vs On-policy 行為解釋
📊 三、穩定性分析：獎勵標準差、掉落懸崖次數/總回合、最差回合獎勵、ε 探索機率計算

【瀏覽器驗證結果】
- Q-learning：收斂 ≈ 151 回合，路徑 14 步（冒險），Std Dev 88.7，掉落 142/500
- SARSA：收斂 ≈ 120 回合，路徑 16 步（保守），Std Dev 22.7，掉落 57/500

三、GitHub 推送與 README

💬 [3] USER：

幫我匯出到 https://github.com/enwu03/0415DRL_HW2.git

🤖 [3] AGENT：

已完成以下操作：
1. 參考 0304DRL_HW1 的 README 風格，撰寫了完整的 README.md
   - 包含演算法公式、參數表格、分析說明、Live Demo 連結、執行方式
2. 複製 demo.webp 操作錄影至專案目錄
3. 執行 git init → git add → git commit → git push 推送至遠端
4. 上傳檔案：README.md, index.html, style.css, script.js, demo.webp

💬 [4] USER：

Live Demo 把內部網址改成這個 https://enwu03.github.io/0415DRL_HW2/

🤖 [4] AGENT：

程式碼與內部網址都沒有問題（CSS/JS 均使用相對路徑），問題是 GitHub Pages 尚未啟用。

請手動操作：
1. 前往 https://github.com/enwu03/0415DRL_HW2/settings/pages
2. Build and deployment → Branch → 選 main，folder 保持 / (root)
3. 點 Save

等 1-2 分鐘部署完成後即可使用。

四、OpenSpec 安裝與初始化

💬 [5] USER：

install openspec v1.1.1

🤖 [5] AGENT：

偵測到系統環境缺少 Node.js（npm 無法使用）。

【執行步驟】
1. 嘗試 pip install openspec → PyPI 上無此套件
2. 搜尋 GitHub → 找到 Fission-AI/OpenSpec，npm 套件名為 @fission-ai/openspec
3. 嘗試 winget install Node.js → 卡在 UAC 管理員提示
4. 改為下載 Node.js v20.12.0 免安裝版 (Portable ZIP)
5. 解壓縮至專案目錄下 node/ 資料夾
6. 透過 npx 成功安裝 @fission-ai/openspec@1.1.1
7. 驗證：openspec --version → 1.1.1 ✓
8. 將 Node.js 路徑加入使用者 PATH 環境變數

[6] USER：

openspec init

[6] AGENT：

執行 openspec init --tools antigravity

【結果】
- 在專案內建立了 .agent/ 目錄
- 匯入 10 個 Skills 和 10 個 Slash Commands（/opsx:new, /opsx:apply, /opsx:archive 等）
- OpenSpec 結構建立完成，可使用 SDD（Spec-driven Development）工作流程

五、handover.md 與圖檔整理

[7] USER：

我已經完成 RL 演算法了。請幫我初始化 OpenSpec，並根據我目前的代碼產出，將作業要求的『第五點：理論比較』與『第六點：結論』寫入 handover.md，並幫我把實驗結果圖檔加上 01- 開頭的編號。

[7] AGENT：

【handover.md 內容】
第五點：理論比較
- Q-learning（Off-policy）：使用 max_a' Q(s',a') 更新，學習「最佳策略」而非「行為策略」
  → 學到沿懸崖最短路徑，但探索時容易掉落
- SARSA（On-policy）：使用實際下一步動作 Q(s',a') 更新，包含探索風險
  → 學到遠離懸崖的安全路徑，穩定性高

第六點：結論
1. 安全性：SARSA 適合高風險環境（自動駕駛、醫療等）
2. 最優解：Q-learning 在 ε=0 部署時提供更高效率
3. 穩定度：SARSA Std Dev 遠低於 Q-learning

【圖檔重新命名】
- 01-training_results.png（學習曲線截圖）
- 01-analysis_1.png（策略行為分析）
- 01-analysis_2.png（穩定性分析）
- 01-demo.webp（操作錄影）

💬 [8] USER：

handover.md 幫我輸出到 https://github.com/enwu03/0415DRL_HW2/tree/main

🤖 [8] AGENT：

已執行 git add → git commit → git push
成功將 handover.md 與 01- 前綴圖檔推送至 GitHub main 分支。

六、tasks.md 與 dev:ending 腳本

💬 [9] USER：

我已經準備好 RL 實驗的檔案。請幫我：
1. 初始化或更新 tasks.md，將『實作 Q-learning 與 SARSA』、『生成比較分析圖』標記為已完成
2. 設定 dev:ending 指令，執行時自動將 handover.md 的內容與目前的 Git commit 關聯
3. 確保 tasks.md 能顯示我的開發時間軸

🤖 [9] AGENT：

【tasks.md】
建立了含有開發時間軸的任務清單：
• 2026-04-15 13:00：完成演算法開發與 UI 視覺化
• 2026-04-15 13:10：完成分析圖檔與 README
• 2026-04-15 13:15：推送至 GitHub
• 2026-04-15 13:20：建立 OpenSpec 交接機制
• [x] 實作 Q-learning 與 SARSA
• [x] 生成比較分析圖

【package.json — dev:ending 腳本】
定義了 npm run dev:ending 指令：
→ 自動 git add handover.md tasks.md → git commit 同步交接紀錄

七、最終整理與推送

💬 [10] USER：

幫我整理到 github，要在 README 寫到我用 OpenSpec 製作部分

[10] AGENT：

已在 README.md 底部新增「Development Workflow」區塊：
說明本專案使用 OpenSpec (@fission-ai/openspec) 進行 Spec-driven Development (SDD)
並將 .agent/ 目錄、tasks.md、package.json 一併推送至 GitHub。

Commit：docs: add OpenSpec workflow documentation and tasks
共 23 個檔案變更（含 10 個 skills + 10 個 workflows）

[11] USER：

打包到桌面，資料夾命名 HW

[11] AGENT：

已將最新版本的專案檔案打包複製至：
C:\Users\student\Desktop\HW\

[12] USER：

幫我生成我的對話 AGENT 的 WORD 檔案放到 "C:\Users\student\Desktop\強化\HW2"

[12] AGENT：

已使用 python-docx 生成完整的 Agent 對話紀錄 Word 文件，
儲存至 C:\Users\student\Desktop\強化\HW2\Agent_Conversation.docx